Introduction to Internet Computing

By Lasitha Gunawardena



What is Internet?

- Internet is a **global communication system** that **links together thousands of individual networks**. It allows exchange of information between two or more computers on a network. Thus internet helps in transfer of messages through mail, chat, video & audio conference, etc. It has become mandatory for day-to-day activities: bills payment, online shopping and surfing, tutoring, working, communicating with peers, etc.
- Internet was evolved in **1969**, under the project called **ARPANET (Advanced Research Projects Agency Network)** to connect computers at **different universities and U.S. defence**. Soon after the people from **different backgrounds** such as **engineers**, **scientists**, **students** and **researchers** started using the network for exchanging information and messages.
- In **1990s** the internet working of **ARPANET**, **NSFnet** and other private networks resulted into Internet. Therefore, Internet is a global network of computer networks' . It comprises of millions of computing devices that carry and transfer volumes of information from one device to the other. Desktop computers, mainframes, GPS units, cell phones, car alarms,video game consoles, are connected to the Net

How do i connect to internet?

- Computer
- Connection - Phone Line, Cable, DSL, Wireless, ...
- Modem
- Network Software - TCP/IP
- Application Software - Web Browser, Email, ...
- Internet Service Provider (ISP)



World Wide Web (WWW)

- The **World Wide Web (WWW)** or web is an **internet based service**, which uses common set of rules known as **protocols**, **to distribute documents across the Internet in a standard way**.
- World Wide Web, which is also known as a **Web**, is a **collection of websites or web pages stored in web servers** and **connected to local computers through the internet**.These websites contain **text pages**, **digital images**, **audios**, **videos**, etc. Users can access the content of these sites from any part of the world over the internet using their devices such as **computers**, **laptops**, **cell phones**, etc. The WWW, along with internet, enables the retrieval and display of text and media to your device.
- The World Wide Web. Or **'Web' is a part of the Internet**. The Web is viewed through **web browser** software such as **Google chrome**, **Internet Explorer**, **Mozilla Firefox** etc.Using browsers one can access the digital libraries containing innumerable articles,journals, e-books, news, tutorials stored in the form of web pages on computers around the world called web servers-Today thousands of web pages/websites are added to the WWW every hour.
-

WWW

- The Web was invented in 1991 by **Tim Berners-Lee**, while consulting at **CERN**
- **(European Organization for Nuclear Research)** in Switzerland.
- The Web is a distributed information system.
- The Web contains multimedia.
- Information in the Web is connected by hyperlinks.

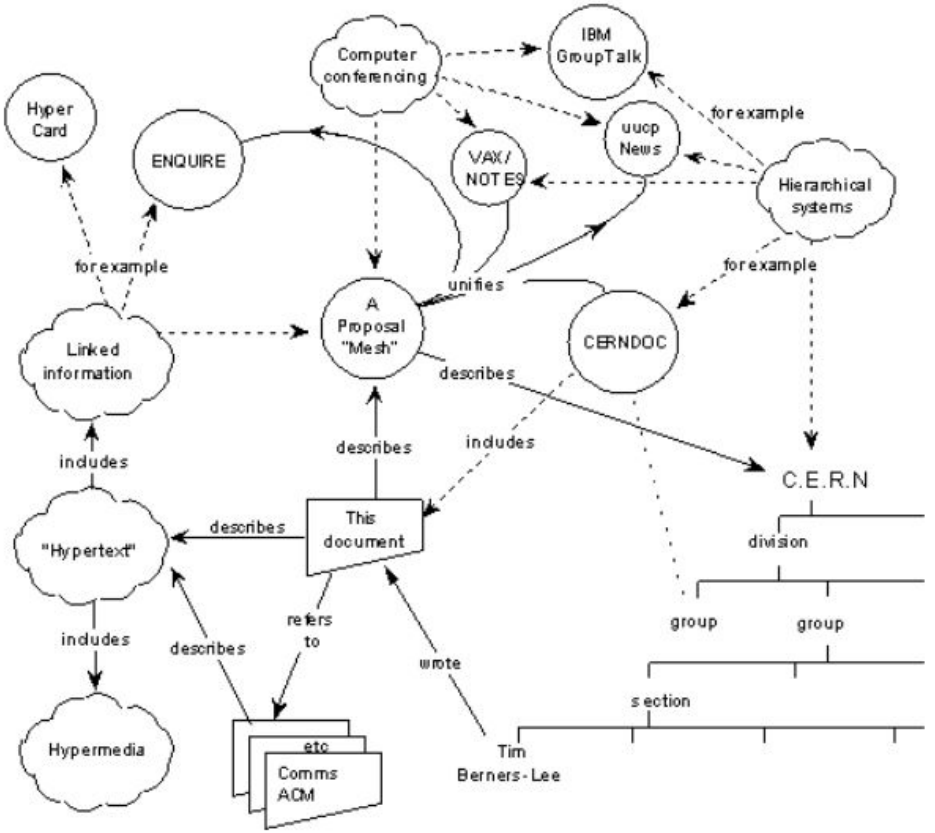
WWW

- In 1989, Tim Berners-Lee had suggested a way to let all users, but particularly scientists, browse each others' papers on the Internet
- He developed HTML, URLs,and HTTP.



[107] **Tim Berners-Lee,** the father of the World Wide Web.

TBL's Web architecture



Internet Vs WWW



Connection Devices

Connecting People

What is a network protocol?

- In **networking**, a **protocol** is a **set of rules** for **formatting** and **processing data**. **Network protocols are like a common language for computers**. The computers within a network may use vastly different software and hardware; however, the use of **protocols enables them to communicate with each other** regardless.
- Standardized protocols are like a common language that computers can use, similar to how two people from different parts of the world may not understand each other's native languages, but they can communicate using a shared third language. If one computer uses the **Internet Protocol (IP)** and a second computer does as well, they will be able to communicate — just as the United Nations relies on its 6 official languages to communicate amongst representatives from all over the globe. But if one computer uses IP and the other does not know this protocol, they will be unable to communicate.
- On the **Internet**, there are **different protocols** for different **types of processes**. Protocols are often discussed in terms of which **OSI model layer** they belong to.

OSI model

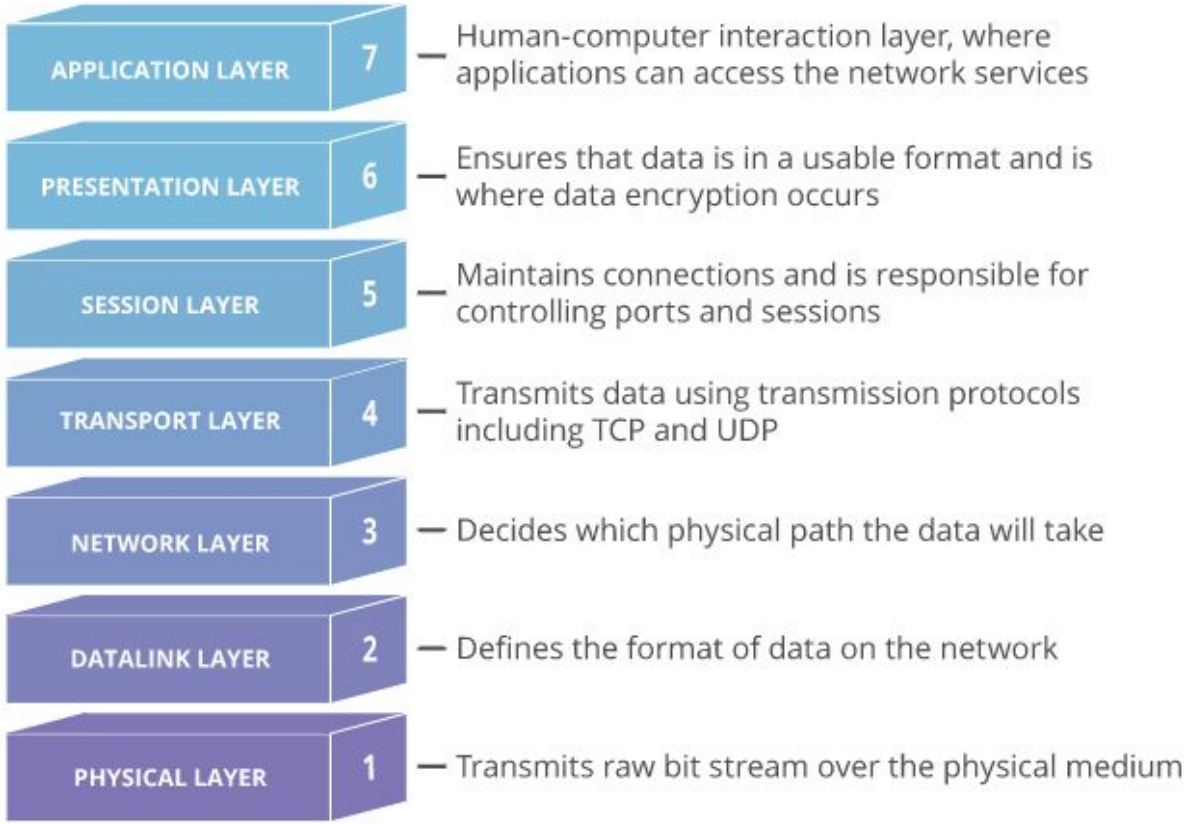
The **Open Systems Interconnection (OSI)** model is an abstract representation of how the Internet works. It contains **7 layers**, with each layer representing a different category of networking functions.

Protocols make these networking functions possible. For instance, the **Internet Protocol (IP)** is responsible for routing data by indicating where data packets* come from and what their destination is. IP makes network-to-network communications possible. Hence, IP is considered a network layer (layer 3) protocol.

As another example, the **Transmission Control Protocol (TCP)** ensures that the transportation of packets of data across networks goes smoothly. Therefore, TCP is considered a transport layer (layer 4) protocol.

**A packet is a small segment of data; all data sent over a network is divided into packets.*

OSI Model


APPLICATION LAYER
7
Human-computer interaction layer, where applications can access the network services
PRESENTATION LAYER
6
Ensures that data is in a usable format and is where data encryption occurs
SESSION LAYER
5
Maintains connections and is responsible for controlling ports and sessions
TRANSPORT LAYER
4
Transmits data using transmission protocols including TCP and UDP
NETWORK LAYER
3
Decides which physical path the data will take
DATALINK LAYER
2
Defines the format of data on the network
PHYSICAL LAYER
1
Transmits raw bit stream over the physical medium

Protocols run on the network layer

As described above, IP is a network layer protocol responsible for routing. But it is not the only network layer protocol.

IPsec: Internet Protocol Security (**IPsec**) sets up encrypted, authenticated IP connections over a **virtual private network (VPN)**. Technically IPsec is not a protocol, but rather a collection of protocols that includes the **Encapsulating Security Protocol (ESP)**, **Authentication Header (AH)**, and **Security Associations (SA)**.

ICMP: The **Internet Control Message Protocol (ICMP)** reports errors and provides status updates. For example, if a router is unable to deliver a packet, it will send an **ICMP message** back to the packet's source.

IGMP: The **Internet Group Management Protocol (IGMP)** sets up **one-to-many network connections**. IGMP helps set up multicasting, meaning multiple computers can receive data packets directed at one IP address.

Other important protocols

TCP: As described above, TCP is a transport layer protocol that ensures reliable data delivery. TCP is meant to be used with IP, and the two protocols are often referenced together as TCP/IP.

HTTP: The **Hypertext Transfer Protocol** (HTTP) is the **foundation** of the **World Wide Web**, the Internet that most users interact with. It is used for transferring data between devices. HTTP belongs to the **application layer** (layer 7), because it puts data into a format that applications (e.g. a browser) can use directly, without further interpretation. The lower layers of the OSI model are handled by a computer's operating system, not applications.

HTTPS: The problem with HTTP is that it is **not encrypted** — any attacker who intercepts an HTTP message can read it. **HTTPS (HTTP Secure)** corrects this by encrypting HTTP messages.

TLS/SSL: Transport Layer Security (TLS) is the protocol **HTTPS** uses for **encryption**. TLS used to be called **Secure Sockets Layer (SSL)**.

UDP: The **User Datagram Protocol (UDP)** is a **faster** but **less reliable alternative** to **TCP** at the transport layer. It is often used in services like **video streaming** and **gaming**, where **fast data delivery is paramount**.

Software Architecture and its types

Before going to explain the types of architecture firstly you need to understand the different layers which are the foundation of software architecture. There are four types of layer:

Presentation Layer: This layer is responsible to display the user interface and manage user interaction.

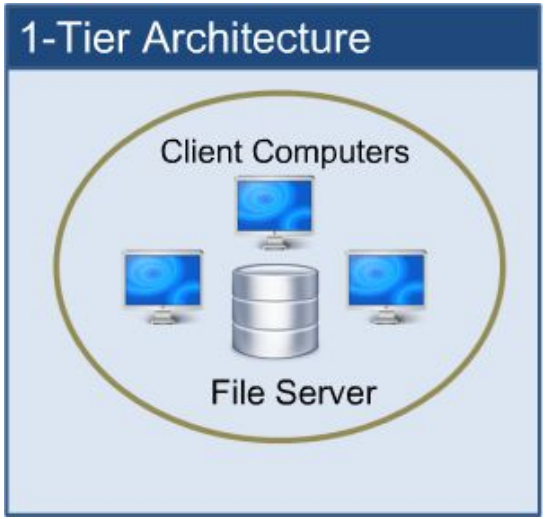
Application Layer: Application layer (also known as Business Layer) has all the business logic, rules and policies. The application layer is a bridge between the presentation layer and the data layer.

Data Layer: This layer is responsible for storing the data.

Service Layer: This layer is responsible to define and implement the service interface and the data contracts. Service layer communicates with the application layer

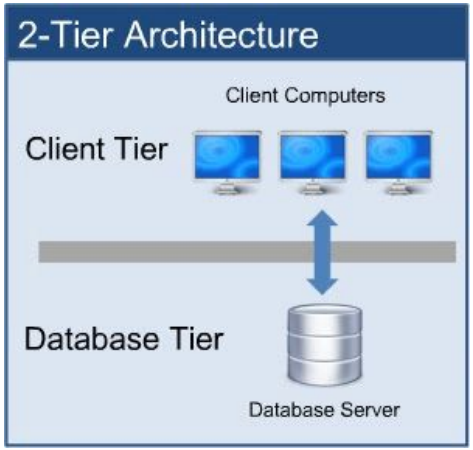
One-tier Architecture

One-tier architecture has **Presentation layer**, **Business layer** and **Data layers** at the same tier i.e. at **Client Tier**. As the name suggested, all the layers and components are available on the **same machine**. **MP3 player**, **MS Office** etc. are some of the examples of one-tier architecture. To store the data (as a function of Data Layer) local system or a shared drive is used.



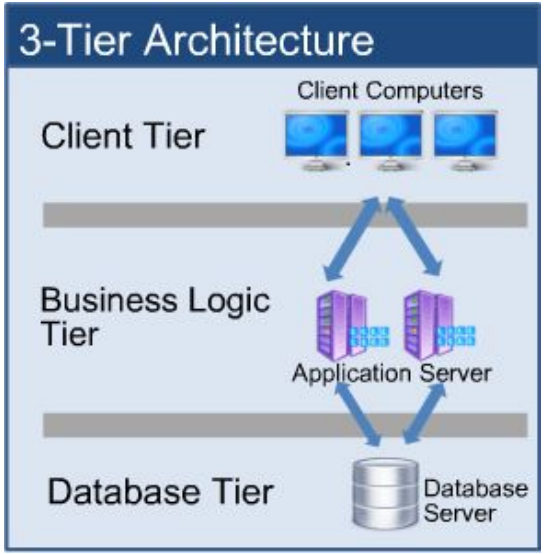
Two-tier Architecture

In such type of architecture, the **client tier** handles both **Presentation** and **Application layers** and the server handles the **Database layer**. The two-tier architecture is also known as a '**Client-Server Application**'. In two-tier architecture, communication takes place between the **Client** and the **Server**. Client system sends the request to the server system and the server system processes the request and sends the response back to the client system.



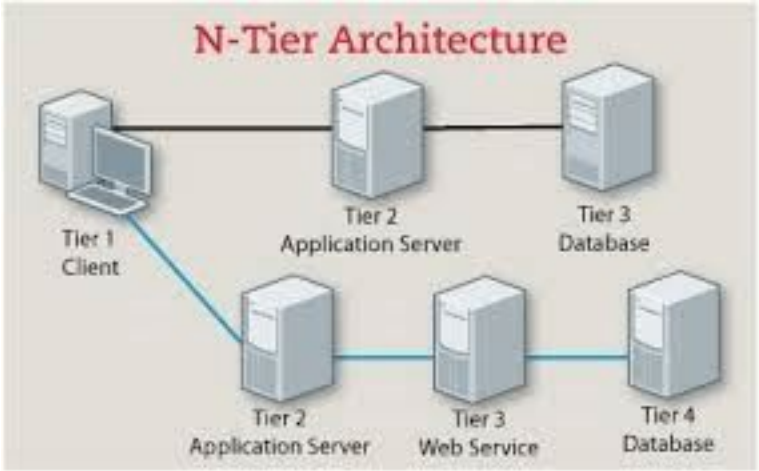
Three-tier Architecture

All three major layers are separated from each other. **Presentation layer** resides at **Client Tier**, **Application layer** acts as **middle-ware** and lies at **Business Tier** and **Data layer** is available at **Data Tier**. This is a very **common architecture**.



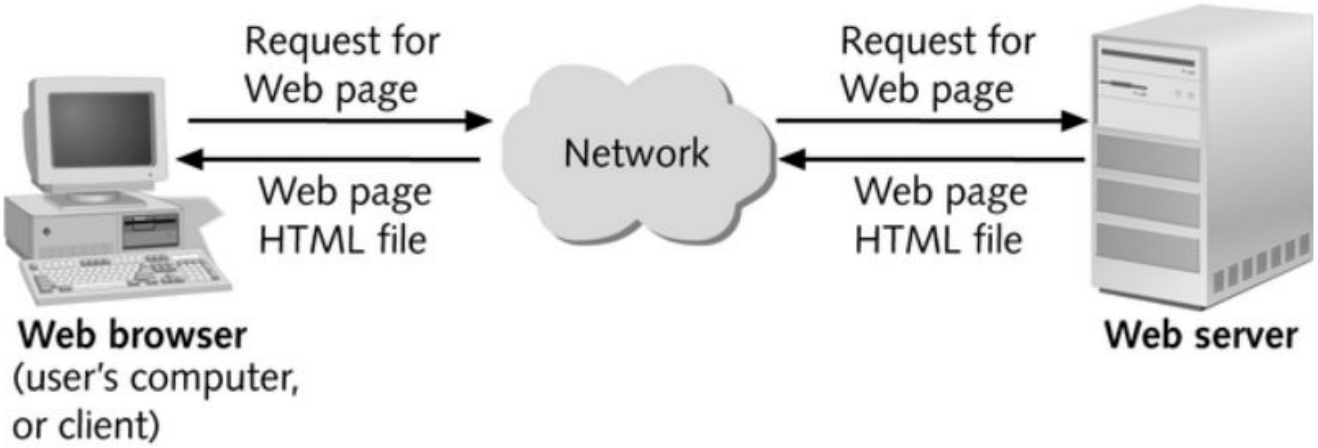
N-tier Architecture

N-tier architecture is also called a **Distributed Architecture** or **Multi-tier Architecture**. It is similar to **three-tier architecture** but the **number** of the **application server** is **increased** and **represented in individual tiers** in order to **distribute** the **business logic** so that the **logic** can be **distributed**.



Basic Web Architecture

- The web is a 2-tiered architecture.
- A web browser displays information content,
- A web server that transfers information to the client.



Web Browser

- The primary purpose is to bring information resources to the user.
- An application for retrieving,presenting, and traversing information resources.



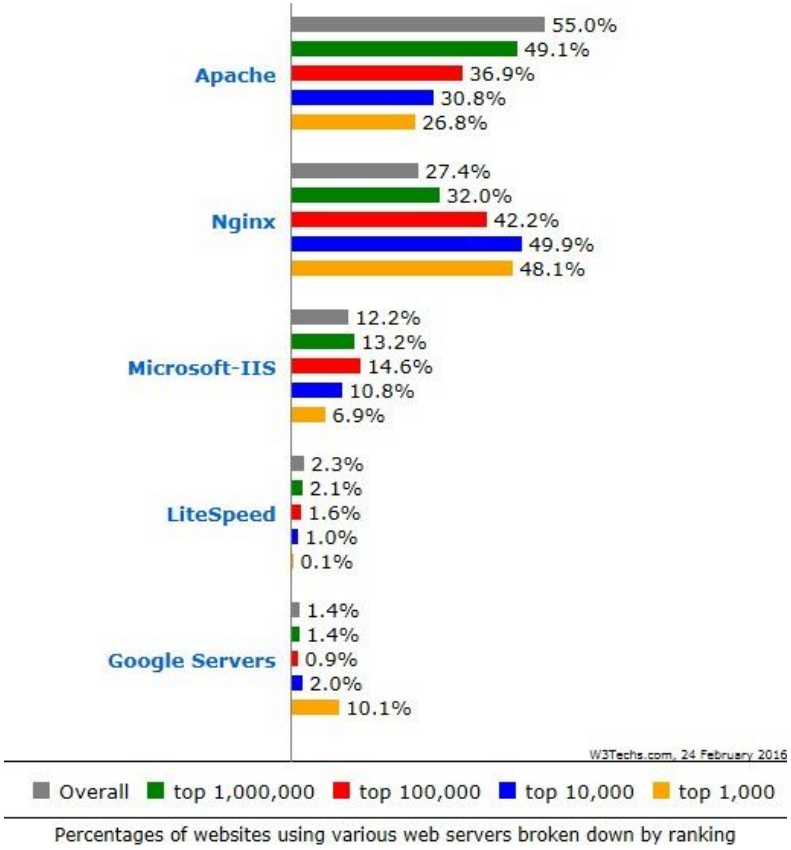
Web Server

The term web server or webserver can mean one of two things:

1. A computer program that accepts HTTP requests and return HTTP responses with optional data content.
2. A computer that runs a computer program as described above.



The best web server



Apache http server

- The Apache HTTP Server was created by Robert McCool in 1995 and has been developed under the direction of the Apache Software Foundation since 1999. Since the HTTP web server is the foundation’s original project and is by far their most popular piece of software, it is often referred to simply as “Apache”.
- The Apache web server was the most popular server on the internet from at least 1996 through 2016. Because of this popularity, Apache benefits from great documentation and integrated support from other software projects.
- Apache is often chosen by administrators for its flexibility, power, and near-universal support. It is extensible through a dynamically loadable module system and can directly serve many scripting languages, such as PHP, without requiring additional software.

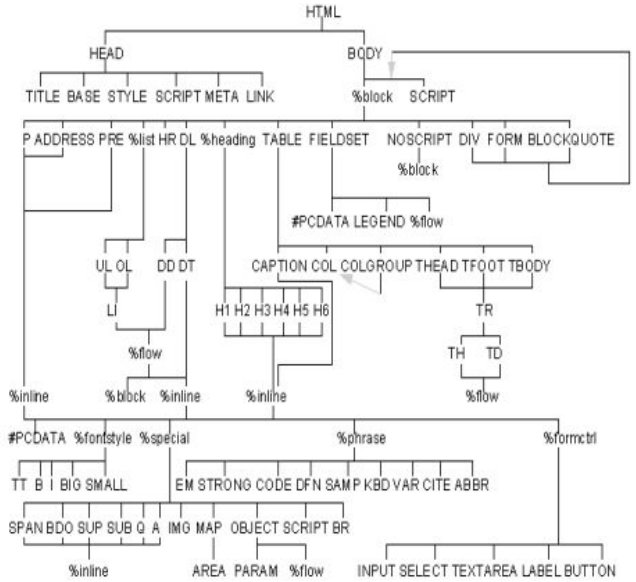
Nginx

- In 2002, Igor Sysoev began work on Nginx as an answer to the **C10K problem**, which was an outstanding challenge for web servers to be able to handle ten thousand concurrent connections. Nginx was publicly released in 2004, and met this goal by relying on an asynchronous, events-driven architecture.
- Nginx has since surpassed Apache in popularity due to its lightweight footprint and its ability to scale easily on minimal hardware. Nginx excels at serving static content quickly, has its own robust module system, and can proxy dynamic requests off to other software as needed.
- Nginx is often selected by administrators for its resource efficiency and responsiveness under load, as well as its straightforward configuration syntax.



HTML

- Document layout language (not a programming language)
- Defines structure and appearance of Web pages



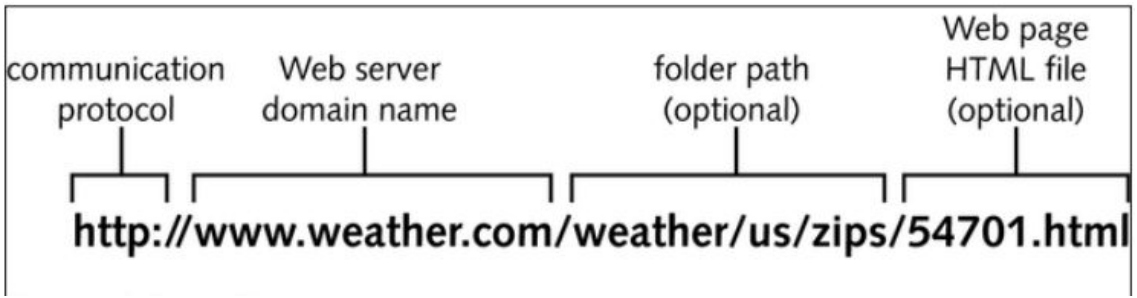
Universal Resource Identifier (URI)

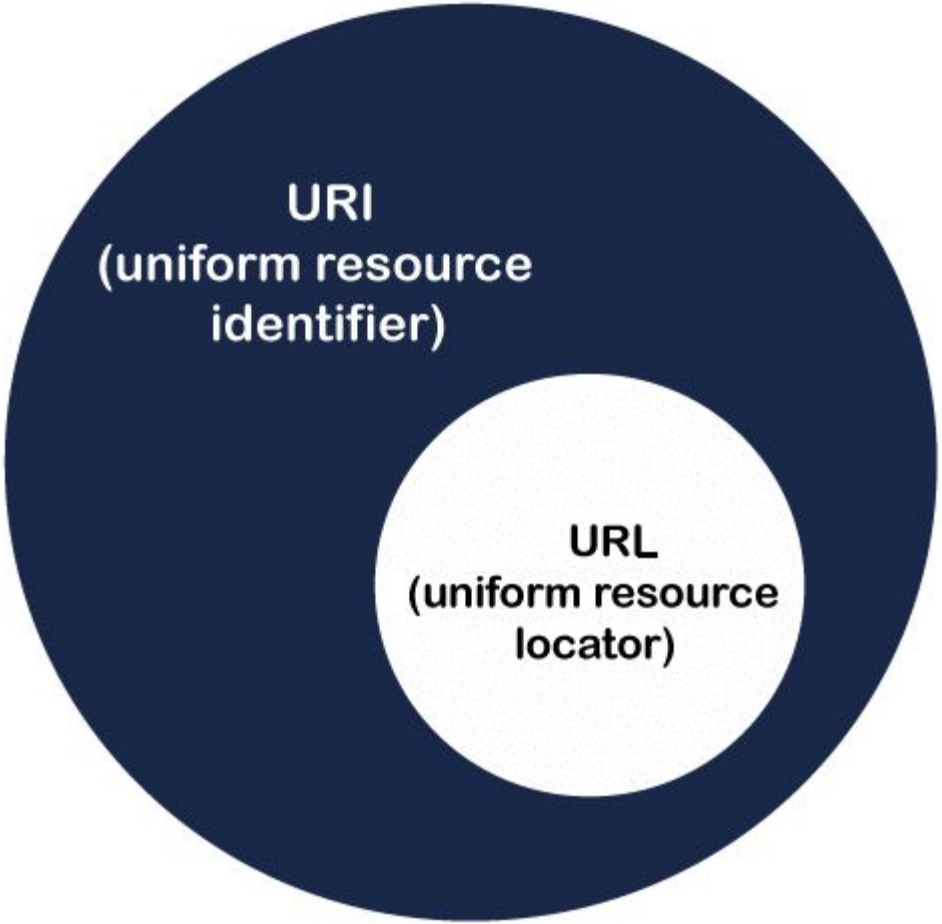
A URI or Uniform Resource Identifier is a **string identifier** that refers to a **resource** on the **internet**. It is a **string of characters** that is used to identify any resource on the internet using **location**, **name**, or both.

1. mailto:hey.Doe@example.com
2. news:comp.infosystems.www.servers.unix
3. urn:oasis:names:specification:docbook:dtd:xml:4.1.2

Uniform Resource Locator (URL)

- A URL or Uniform Resource Locator is used to find the location of the resource on the web. It is a **reference** for a **resource** and a way to **access that resource**. **A URL always shows a unique resource**, and it can be an HTML page, a CSS document, an image, etc.
- A URL uses a protocol for accessing the resource, which can be **HTTP**, **HTTPS**, **FTP**, etc.
- It is mainly referred to as the address of the website, which a user can find in their address bars.




URI
(uniform resource
identifier)
URL
(uniform resource
locator)

HyperText Transfer Protocol (HTTP)

- HTTP is a request/response standard of a client and a server.
- Typically, an HTTP client initiates a request.
- Resources to be accessed by HTTP are identified using Uniform Resource Identifiers (URIs).

Basic HTTP Features

Basic Features

There are three basic features that make HTTP a simple but powerful protocol:

HTTP is connectionless: The HTTP client, i.e., a browser initiates an HTTP request and after a request is made, the client waits for the response. The server processes the request and sends a response back after which client disconnect the connection. So client and server knows about each other during current request and response only. Further requests are made on new connection like client and server are new to each other.

HTTP is media independent: It means, any type of data can be sent by HTTP as long as both the client and the server know how to handle the data content. It is required for the client as well as the server to specify the content type using appropriate MIME-type.

HTTP is stateless: As mentioned above, HTTP is connectionless and it is a direct result of HTTP being a stateless protocol. The server and client are aware of each other only during a current request. Afterwards, both of them forget about each other. Due to this nature of the protocol, neither the client nor the browser can retain information between different requests across the web pages.

HTTP request anatomy

An HTTP client sends an HTTP request to a server in the form of a request message which includes following format:

- A Request-line
- Zero or more header (General|Request|Entity) fields followed by CRLF
- An empty line (i.e., a line with nothing preceding the CRLF) indicating the end of the header fields
- Optionally a message-body

Request-Line

The Request-Line begins with a **method token**, followed by the **Request-URI** and the **protocol version**, and ending with CRLF. The elements are separated by space SP characters.

CRLF - **Carriage Return** and **Line Feed**

Request-Line = [Method][SP] [Request-URI][SP] [HTTP-Version] CRLF

Request Method

The request method indicates the method to be performed on the resource identified by the given **Request-URI**. The method is **case-sensitive** and **should always** be mentioned in **uppercase**.

HTTP /1.1 Methods

S.N.	Method and Description
1	**GET** The GET method is used to retrieve information from the given server using a given URI. Requests using GET should only retrieve data and should have no other effect on the data.
2	**HEAD** Same as GET, but it transfers the status line and the header section only.
3	**POST** A POST request is used to send data to the server, for example, customer information, file upload, etc. using HTML forms.
4	**PUT** Replaces all the current representations of the target resource with the uploaded content.
5	**DELETE** Removes all the current representations of the target resource given by URI.
6	**CONNECT** Establishes a tunnel to the server identified by a given URI.
7	**OPTIONS** Describe the communication options for the target resource.
8	**TRACE** Performs a message loop back test along with the path to the target resource.

Request Header Fields

The request-header fields allow the client to pass **additional information** about the request, and about the client itself, to the server. These fields act as request modifiers.Here is a list of some important Request-header fields that can be used based on the requirement:

- Accept-Charset
- Accept-Encoding
- Accept-Language
- Authorization
- Expect
- From
- Host
- If-Match
- If-Modified-Since
- If-None-Match
- If-Range
- If-Unmodified-Since
- Max-Forwards
- Proxy-Authorization
- Range
- Referer
- TE
- User-Agent

Examples of Request Message

Sample request made to the hello.html

```
GET /hello.htm HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.tutorialspoint.com
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive
```

HTTP response anatomy

After receiving and interpreting a request message, a server responds with an HTTP response message:

- A Status-line
- Zero or more header (General|Response|Entity) fields followed by CRLF
- An empty line (i.e., a line with nothing preceding the CRLF)
  indicating the end of the header fields
- Optionally a message-body

Message Status-Line

A Status-Line consists of the protocol version followed by a numeric status code and its associated textual phrase. The elements are separated by space SP characters.

Status-Line = [HTTP-Version][SP][Status-Code][SP][Reason-Phrase] CRLF

HTTP Version

A server supporting HTTP version 1.1 will return the following version information:

HTTP-Version = HTTP/1.1

Status Code

The Status-Code element is a **3-digit integer** where first digit of the **Status-Code** defines the class of response and the **last two digits do not have any categorization role**. There are 5 values for the first digit:

S.N.	Code and Description
1	**1xx: Informational** It means the request was received and the process is continuing.
2	**2xx: Success** It means the action was successfully received, understood, and accepted.
3	**3xx: Redirection** It means further action must be taken in order to complete the request.
4	**4xx: Client Error** It means the request contains incorrect syntax or cannot be fulfilled.
5	**5xx: Server Error** It means the server failed to fulfill an apparently valid request.

Response Header Fields

The response-header fields allow the **server to pass additional information** about the response which cannot be placed in the Status- Line. These header fields give information about the server and about further access to the resource identified by the Request-URI.

- Accept-Ranges
- Age
- ETag
- Location
- Proxy-Authenticate
- Retry-After
- Server
- Vary
- WWW-Authenticate

Examples of Response Message

```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
Content-Length: 88
Content-Type: text/html
Connection: Closed
```

Run NGINX

1. Download the nginx latest server from https://nginx.org/
2. Make sure you don't have a running server in your PC on port 80. You can check this from your browser whether you have a web page running in http://localhost.
3. If its IIS you can stop it from command follow as administrator

```
/> iisreset /stop
```

4. Unzip nginx downloaded zip bundle to a directory. (Ex c:/nginx-1.22)
5. Now execute nginx.exe
6. When you access http://localhost from the browser you can see nginx is running.
7. To stop nginx service go to task manage and kill the service.

About Me

Create a simple HTML file called aboutme.html. Then copy the file to [NGINX_HOME]/html.Then try to access it from the browser (http://localhost/aboutme.html)

```
<!DOCTYPE html>
<html>
	<head>
		<title>About Me</title>
	</head>
	<body>
		<h1>About Me</h1>
		<p><b>Name:</b> Lasitha Gunawardena</p>
	</body>
</html>
```